UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Squire's Ascent Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Squire's Ascent

Squire's Ascent is a standalone editable Unity game template. Time advances while your selected job pays crowns and gains experience. Train a skill alongside the job, select newly opened careers and manage housing and goods. Follow chapter goals and the supplied Ages progression; all progression is local.

Includes 25 curated chapters, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Additional included libraries: 9 categories, 20 jobs, 17 skills, 16 items, 4 boosters, 4 seasons. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Squire's Ascent > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/SquireAscent/Scenes/SquireAscent_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/SquireAscent`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/squireascent contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/SqAssets.cs`
- `Scripts/SqData.cs`
- `Scripts/SqEngine.cs`
- `Scripts/SqGen.cs`
- `Scripts/SquireAscentGame.cs`
- `Scripts/SquireAscentGame.Layout.cs`
- `Scripts/SquireAscentGame.Popups.cs`
- `Scripts/SquireAscentGame.Tutorial.cs`

Main flow: SquireAscentGame.cs. UI and popup consumers: SquireAscentGame.Popups.cs. Model/generator/loading: SqData.cs, SqEngine.cs, SqGen.cs. Authoring: Editor/SquireAscentLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Time advances while your selected job pays crowns and gains experience. Train a skill alongside the job, select newly opened careers and manage housing and goods. Follow chapter goals and the supplied Ages progression; all progression is local.

The initial bank has 25 curated chapters. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact helper names, unlocks, charges and effects: Resources/squireascent/Levels/levels.json / boosters.

Helper	Unlock	Free charges	Action and effect
Coin Purse	3	3	Instantly grants about 120 days of your current pay.
Time Warp	6	3	Doubles the speed of time for 60 seconds.
Focus Candle	10	3	Triples skill XP for 60 seconds.
Herald's Writ	15	2	For 3 minutes, promotes you to the best open job in your track and trains your lowest skill.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Squire's Ascent > Level Editor**. Exact implementation: `Editor/SquireAscentLevelEditor.cs`.

Duplicate an early entry in this game-specific visual editor, change its visible name and one rule, save, validate, then play it. Use the exact source-control table below for this game.

1. Back up Resources/squireascent/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`SquireAscentLevelEditor.cs:43`	`[MenuItem("Szekipapa77/Squire's Ascent/Level Editor")]`
`SquireAscentLevelEditor.cs:46`	`[MenuItem("Szekipapa77/Squire's Ascent/Validate All Levels")]`
`SquireAscentLevelEditor.cs:104`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`SquireAscentLevelEditor.cs:105`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`SquireAscentLevelEditor.cs:108`	`if (GUILayout.Button(new GUIContent("Validate", "Bot plays to the selected chapter and reports its time"), EditorStyles.toolbarButton, GUILayout.Width(70))) RunValidation(_sel + 1);`
`SquireAscentLevelEditor.cs:109`	`if (GUILayout.Button(new GUIContent("Validate All", "Bot plays every curated chapter plus 8 generated ones"), EditorStyles.toolbarButton, GUILayout.Width(85))) RunValidation(_file.chapters.Length + 8);`

SquireAscentLevelEditor.cs:114	if (GUILayout.Button(new GUIContent("Quick test", "Optional: saves, opens the WebGL scene at this chapter and enters Play mode. Editing never needs it."), EditorStyles.toolbarButton, GUILayout.Width(80))) Play(_sel + 1);
SquireAscentLevelEditor.cs:145	if (GUILayout.Button($"{i + 1,2}. {c.title} {(c.breather ? " [b]" : "")}{time}\n {SqSim.GoalText(c, sim)}", EditorStyles.helpBox, GUILayout.Height(36))) { _sel = i; GUI.FocusControl(null); }
SquireAscentLevelEditor.cs:150	if (GUILayout.Button("Add")) { var l = new List<SqChapter>(_file.chapters) { new SqChapter { title = "New Chapter", kind = "levels", value = 100 } }; _file.chapters = l.ToArray(); _sel = l.Count - 1; _dirty = true; }
SquireAscentLevelEditor.cs:151	if (GUILayout.Button("Duplicate") && Cur != null) { var l = new List<SqChapter> (_file.chapters); l.Insert(_sel + 1, JsonUtility.FromJson<SqChapter> (JsonUtility.ToJson(Cur))); _file.chapters = l.ToArray(); _sel++; _dirty = true; }
SquireAscentLevelEditor.cs:152	if (GUILayout.Button("Delete") && Cur != null) { var l = new List<SqChapter>(_file.chapters); l.RemoveAt(_sel); _file.chapters = l.ToArray(); _sel = Mathf.Max(0, _sel - 1); _dirty = true; }
SquireAscentLevelEditor.cs:153	if (GUILayout.Button("Up") && _sel > 0) { Swap(_sel, _sel - 1); _sel--; }
SquireAscentLevelEditor.cs:154	if (GUILayout.Button("Down") && _sel < _file.chapters.Length - 1) { Swap(_sel, _sel + 1); _sel++; }
SquireAscentLevelEditor.cs:163	EditorGUILayout.LabelField($"Chapter {_sel + 1}", EditorStyles.boldLabel);
SquireAscentLevelEditor.cs:164	c.title = EditorGUILayout.TextField("Title", c.title);
SquireAscentLevelEditor.cs:165	c.hint = EditorGUILayout.TextField("Hint (first-time text)", c.hint);
SquireAscentLevelEditor.cs:167	k = EditorGUILayout.Popup("Goal kind", k, Kinds);

SquireAscentLevelEditor.cs:173	`if (ids.Count > 0) c.id = ids[EditorGUILayout.Popup(c.kind == "job" ? "Job" : "Skill", i, ids.ToArray())];`
SquireAscentLevelEditor.cs:174	`c.value = EditorGUILayout.IntField("Level (1 = work as)", (int)c.value);`
SquireAscentLevelEditor.cs:180	`if (ids.Count > 0) c.id = ids[EditorGUILayout.Popup("Home", i, ids.ToArray())];`
SquireAscentLevelEditor.cs:183	`else c.value = EditorGUILayout.DoubleField("Target value", c.value);`
SquireAscentLevelEditor.cs:184	`c.breather = EditorGUILayout.Toggle("Breather", c.breather);`
SquireAscentLevelEditor.cs:187	`bi = EditorGUILayout.Popup("Bonus booster", bi, bids.ToArray());`
SquireAscentLevelEditor.cs:189	`if (bi > 0) c.charges = EditorGUILayout.IntSlider("Bonus charges", c.charges, 0, 5);`
SquireAscentLevelEditor.cs:190	`foreach (var b in _file.boosters) if (b.unlock == _sel + 1) EditorGUILayout.HelpBox($"Reaching this chapter unlocks the {b.name} booster.", MessageType.Info);`
SquireAscentLevelEditor.cs:191	`EditorGUILayout.HelpBox("Preview: " + SqSim.GoalText(c, new SqSim(_file)), MessageType.None);`
SquireAscentLevelEditor.cs:198	`EditorGUILayout.LabelField("Difficulty curve (bot minutes per chapter; blue = breather)", EditorStyles.boldLabel);`
SquireAscentLevelEditor.cs:229	`EditorGUILayout.LabelField("Auto Generate (bot trajectory → goals on a time curve)", EditorStyles.boldLabel);`
SquireAscentLevelEditor.cs:231	`_genCount = EditorGUILayout.IntSlider("Count", _genCount, 1, 30);`
SquireAscentLevelEditor.cs:232	`_genStart = EditorGUILayout.IntSlider("Start at chapter", _genStart, 1, _file.chapters.Length + 1);`

SquireAscentLevelEditor.cs:233	`_genSeed = EditorGUILayout.IntField("Seed", _genSeed);`
SquireAscentLevelEditor.cs:234	`_genStartGap = EditorGUILayout.Slider("Start gap (min)", _genStartGap, 0.3f, 60);`
SquireAscentLevelEditor.cs:235	`_genEndGap = EditorGUILayout.Slider("End gap (min)", _genEndGap, 0.3f, 90);`
SquireAscentLevelEditor.cs:236	`_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 0, 10);`
SquireAscentLevelEditor.cs:237	`_genReplace = EditorGUILayout.Toggle("Replace range (else insert)", _genReplace);`
SquireAscentLevelEditor.cs:250	`if (GUILayout.Button("Generate")) Generate();`
SquireAscentLevelEditor.cs:414	`if (GUILayout.Button("Add job")) { var l = new List<SqTask>(_file.jobs) { new SqTask { id = "job" + _file.jobs.Length, name = "New Job", cat = _file.jobs.Length > 0 ? _file.jobs[_file.jobs.Length - 1].cat : "", income = 10 } }; _file.jobs = l.ToArray(); _selTask = l.Count - 1; _dirty = true; }`
SquireAscentLevelEditor.cs:415	`if (GUILayout.Button("Add skill")) { var l = new List<SqTask>(_file.skills) { new SqTask { id = "skill" + _file.skills.Length, name = "New Skill", cat = _file.skills.Length > 0 ? _file.skills[_file.skills.Length - 1].cat : "", stat = "xp", target = "all" } }; _file.skills = l.ToArray(); _selTask = sim.NJ + l.Count - 1; _dirty = true; }`
SquireAscentLevelEditor.cs:417	`if (_selTask < 0 || _selTask >= sim.NT) { EditorGUILayout.HelpBox("Select a node in the graph.", MessageType.Info); return; }`
SquireAscentLevelEditor.cs:420	`EditorGUILayout.LabelField(job ? "Job" : "Skill", EditorStyles.boldLabel);`
SquireAscentLevelEditor.cs:421	`t.id = EditorGUILayout.TextField("Id (icon name)", t.id);`
SquireAscentLevelEditor.cs:422	`t.name = EditorGUILayout.TextField("Name", t.name);`

SquireAscentLevelEditor.cs:425	`if (catIds.Count > 0) t.cat = catIds[EditorGUILayout.Popup("Category", ci, catIds.ToArray())];`
SquireAscentLevelEditor.cs:426	`EditorGUILayout.LabelField("Description");`
SquireAscentLevelEditor.cs:428	`t.maxXp = EditorGUILayout.DoubleField("Base XP (level 0→1)", t.maxXp);`
SquireAscentLevelEditor.cs:429	`if (job) t.income = EditorGUILayout.DoubleField("Base pay / day", t.income);`
SquireAscentLevelEditor.cs:432	`t.stat = Stats[EditorGUILayout.Popup("Stat", Mathf.Max(0, Array.IndexOf(Stats, t.stat)), Stats)];`
SquireAscentLevelEditor.cs:433	`t.target = EditorGUILayout.TextField("Target (all/jobs/skills/cat/id)", t.target);`
SquireAscentLevelEditor.cs:434	`t.curve = Curves[EditorGUILayout.Popup("Curve", Mathf.Max(0, Array.IndexOf(Curves, t.curve)), Curves)];`
SquireAscentLevelEditor.cs:435	`if (t.curve == "lin") t.effect = EditorGUILayout.DoubleField("Effect per level", t.effect);`
SquireAscentLevelEditor.cs:436	`else t.logBase = EditorGUILayout.DoubleField("Log base", t.logBase);`
SquireAscentLevelEditor.cs:437	`EditorGUILayout.LabelField($"Effect at L10 / L100 / L1000: {SqSim.SkillEffect(t, 10):0.00} / {SqSim.SkillEffect(t, 100):0.00} / {SqSim.SkillEffect(t, 1000):0.00}", EditorStyles.miniLabel);`
SquireAscentLevelEditor.cs:439	`EditorGUILayout.LabelField("Requirements", EditorStyles.boldLabel);`
SquireAscentLevelEditor.cs:447	`if (r.kind == "task") r.id = ids[EditorGUILayout.Popup(Mathf.Max(0, ids.IndexOf(r.id)), ids.ToArray())];`
SquireAscentLevelEditor.cs:449	`if (GUILayout.Button("x", GUILayout.Width(22))) { reqs.RemoveAt(i); _dirty = true; EditorGUILayout.EndHorizontal(); break; }`

SquireAscentLevelEditor.cs:452	if (GUILayout.Button("Add requirement")) { reqs.Add(new SqReq { kind = "task", id = ids[0], value = 10 }); _dirty = true; }
SquireAscentLevelEditor.cs:455	if (GUILayout.Button("Delete this " + (job ? "job" : "skill")) && EditorUtility.DisplayDialog("Delete", $"Delete {t.name}?", "Delete", "Cancel"))
SquireAscentLevelEditor.cs:467	EditorGUILayout.LabelField("Shop items (home = pick one, multiplies comfort; goods = toggles)", EditorStyles.boldLabel);
SquireAscentLevelEditor.cs:476	GUILayout.Label("cost", GUILayout.Width(30)); it.expense = EditorGUILayout.DoubleField(it.expense, GUILayout.Width(80));
SquireAscentLevelEditor.cs:477	GUILayout.Label("x", GUILayout.Width(10)); it.effect = EditorGUILayout.DoubleField(it.effect, GUILayout.Width(50));
SquireAscentLevelEditor.cs:481	if (GUILayout.Button("x", GUILayout.Width(22))) { l.RemoveAt(i); _dirty = true; EditorGUILayout.EndHorizontal(); break; }
SquireAscentLevelEditor.cs:484	if (GUILayout.Button("Add item")) { l.Add(new SqItem { id = "item" + l.Count, name = "New Item" }); _dirty = true; }
SquireAscentLevelEditor.cs:490	EditorGUILayout.LabelField("Boosters (unlock one at a time by chapter)", EditorStyles.boldLabel);
SquireAscentLevelEditor.cs:497	GUILayout.Label("unlock ch", GUILayout.Width(62)); b.unlock = EditorGUILayout.IntField(b.unlock, GUILayout.Width(40));
SquireAscentLevelEditor.cs:498	GUILayout.Label("charges", GUILayout.Width(52)); b.charges = EditorGUILayout.IntField(b.charges, GUILayout.Width(40));
SquireAscentLevelEditor.cs:499	GUILayout.Label("duration s", GUILayout.Width(64)); b.duration = EditorGUILayout.FloatField(b.duration, GUILayout.Width(50));

SquireAscentLevelEditor.cs:500	GUILayout.Label("power", GUILayout.Width(40)); b.power = EditorGUILayout.FloatField(b.power, GUILayout.Width(40));
SquireAscentLevelEditor.cs:502	b.desc = EditorGUILayout.TextField("Description", b.desc);
SquireAscentLevelEditor.cs:503	b.guide = EditorGUILayout.TextField("Guided first use", b.guide);
SquireAscentLevelEditor.cs:511	EditorGUILayout.LabelField("Time and life", EditorStyles.boldLabel);
SquireAscentLevelEditor.cs:512	c.daysPerSec = EditorGUILayout.DoubleField("Days per second", c.daysPerSec);
SquireAscentLevelEditor.cs:513	c.startAge = EditorGUILayout.DoubleField("Start age", c.startAge);
SquireAscentLevelEditor.cs:514	c.lifespan = EditorGUILayout.DoubleField("Base lifespan (years)", c.lifespan);
SquireAscentLevelEditor.cs:515	c.agesTabAge = EditorGUILayout.DoubleField("Ages tab at age", c.agesTabAge);
SquireAscentLevelEditor.cs:516	c.rebirthAge = EditorGUILayout.DoubleField("New Age from age", c.rebirthAge);
SquireAscentLevelEditor.cs:517	c.ascendAge = EditorGUILayout.DoubleField("Ascend from age", c.ascendAge);
SquireAscentLevelEditor.cs:518	EditorGUILayout.LabelField($"One year = {365 / c.daysPerSec:0.0} s; first life ≈ {(c.lifespan - c.startAge) * 365 / c.daysPerSec / 60:0.0} min", EditorStyles.miniLabel);
SquireAscentLevelEditor.cs:519	EditorGUILayout.LabelField("Economy", EditorStyles.boldLabel);
SquireAscentLevelEditor.cs:520	c.xpPerDay = EditorGUILayout.DoubleField("Base XP per day", c.xpPerDay);
SquireAscentLevelEditor.cs:521	c.laurel = EditorGUILayout.DoubleField("Laurel XP per chapter", c.laurel);

SquireAscentLevelEditor.cs:522	c.shopCoins = EditorGUILayout.DoubleField("Market opens at crowns", c.shopCoins);
SquireAscentLevelEditor.cs:523	c.itemReveal = EditorGUILayout.DoubleField("Item shows at cost ×", c.itemReveal);
SquireAscentLevelEditor.cs:524	c.purseDays = EditorGUILayout.DoubleField("Coin Purse days of pay", c.purseDays);
SquireAscentLevelEditor.cs:525	c.offlineHours = EditorGUILayout.DoubleField("Offline cap (hours)", c.offlineHours);
SquireAscentLevelEditor.cs:526	EditorGUILayout.LabelField("Endless generator", EditorStyles.boldLabel);
SquireAscentLevelEditor.cs:527	c.genSeed = EditorGUILayout.IntField("Seed", c.genSeed);
SquireAscentLevelEditor.cs:528	c.genLevels = EditorGUILayout.DoubleField("Total level base", c.genLevels); c.genLevelsGrow = EditorGUILayout.DoubleField(" growth", c.genLevelsGrow);
SquireAscentLevelEditor.cs:529	c.genIncome = EditorGUILayout.DoubleField("Pay base", c.genIncome); c.genIncomeGrow = EditorGUILayout.DoubleField(" growth", c.genIncomeGrow);
SquireAscentLevelEditor.cs:530	c.genCoins = EditorGUILayout.DoubleField("Crowns base", c.genCoins); c.genCoinsGrow = EditorGUILayout.DoubleField(" growth", c.genCoinsGrow);
SquireAscentLevelEditor.cs:532	for (int n = _file.chapters.Length + 1; n <= _file.chapters.Length + 6; n++) EditorGUILayout.LabelField($" ch {n}: " + SqSim.GoalText(SqGen.Endless(_file, n), sim), EditorStyles.miniLabel);
SquireAscentLevelEditor.cs:533	EditorGUILayout.LabelField("Seasons", EditorStyles.boldLabel);

CHAPTER 07

Generator, insertion and replacement

Use this editor generation panel, choose a reproducible seed and small count, review start/index and replace behavior, then generate, save and reopen. The source excerpt below identifies exact append/replace semantics.

Source	Exact current code
`SquireAscentLevelEditor.cs:40`	`bool _genReplace;`
`SquireAscentLevelEditor.cs:237`	`_genReplace = EditorGUILayout.Toggle("Replace range (else insert)", _genReplace);`
`SquireAscentLevelEditor.cs:265`	`if (_genReplace) list.RemoveRange(start0, Mathf.Min(_genCount, Mathf.Max(0, list.Count - start0)));`
`SquireAscentLevelEditor.cs:266`	`list.InsertRange(Mathf.Min(start0, list.Count), gen);`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/SquireAscent/Resources/squireascent/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
{
  "title": "A Humble Start",
  "hint": "Tap a job to work it. Jobs pay crowns every day and level up over time.",
  "kind": "job",
  "id": "stablehand",
  "value": 5,
  "booster": "",
  "charges": 0,
  "breather": false,
  "minutes": 0.2
}
```


Root keys: config, categories, jobs, skills, items, chapters, boosters, seasons. Inspect SqData.cs, SqEngine.cs, SqGen.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/SquireAscentGame.cs`. Find `Ink`, currently `#3B2A1E`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`SquireAscentGame.cs:24 / static Color X(string h) => HzUI.Hex(h);`	`static readonly Color Parchment = X("#F3E3C3"), Parch2 = X("#E9D3A8"), Ink = X("#3B2A1E"), InkSoft = X("#6B5440"),`
`SquireAscentGame.Layout.cs:269 / void BuildStatusPortrait(RectTransform hud)`	`_jobTitle = T(c, "Stable Hand", 46, Ink, TextAnchor.MiddleLeft, true); TL(_jobTitle.rectTransform, 256, 24, 560, 60);`
`SquireAscentGame.Layout.cs:270 / void BuildStatusPortrait(RectTransform hud)`	`_ageText = T(c, "Age 16", 34, InkSoft); TL(_ageText.rectTransform, 256, 86, 330, 46);`
`SquireAscentGame.Layout.cs:272 / void BuildStatusPortrait(RectTransform hud)`	`_seasonText = T(c, "Spring", 28, InkSoft); TL(_seasonText.rectTransform, 290, 128, 520, 44);`
`SquireAscentGame.Layout.cs:276 / void BuildStatusPortrait(RectTransform hud)`	`_activeLine = T(c, "", 26, InkSoft); TL(_activeLine.rectTransform, 256, 206, 560, 40);`
`SquireAscentGame.Layout.cs:283 / void BuildStatusPortrait(RectTransform hud)`	`_coinsText = T(coinRow.transform, "0", 72, Ink, TextAnchor.MiddleLeft, true); TL(_coinsText.rectTransform, 122, 8, 420, 84);`
`SquireAscentGame.Layout.cs:285 / void BuildStatusPortrait(RectTransform hud)`	`_payText = T(coinRow.transform, "", 28, InkSoft, TextAnchor.MiddleRight); TL(_payText.rectTransform, 560, 12, 410, 40);`
`SquireAscentGame.Layout.cs:286 / void BuildStatusPortrait(RectTransform hud)`	`_comfortText = T(coinRow.transform, "", 28, InkSoft, TextAnchor.MiddleRight); TL(_comfortText.rectTransform, 560, 54, 410, 40);`
`SquireAscentGame.Layout.cs:287 / void BuildStatusPortrait(RectTransform hud)`	`_laurelText = T(coinRow.transform, "", 28, InkSoft, TextAnchor.MiddleRight); TL(_laurelText.rectTransform, 560, 96, 410, 40);`

SquireAscentGame.Layout.cs:303 / void BuildStatus(RectTransform col)	_jobTitle = T(c, "Stable Hand", 34, Ink, TextAnchor.MiddleLeft, true); TL(_jobTitle.rectTransform, 180, 18, 236, 84);
SquireAscentGame.Layout.cs:305 / void BuildStatus(RectTransform col)	_ageText = T(c, "Age 16", 25, InkSoft); TL(_ageText.rectTransform, 180, 100, 236, 32);
SquireAscentGame.Layout.cs:307 / void BuildStatus(RectTransform col)	_seasonText = T(c, "Spring", 19, InkSoft); TL(_seasonText.rectTransform, 206, 134, 210, 34);
SquireAscentGame.Layout.cs:311 / void BuildStatus(RectTransform col)	_activeLine = T(c, "", 22, InkSoft); HBand(_activeLine.rectTransform, 220, 70, 18, 14);
SquireAscentGame.Layout.cs:317 / void BuildStatus(RectTransform col)	_coinsText = T(k, "0", 60, Ink, TextAnchor.MiddleLeft, true); TL(_coinsText.rectTransform, 104, 10, 300, 76);
SquireAscentGame.Layout.cs:319 / void BuildStatus(RectTransform col)	_payText = T(k, "", 22, InkSoft); TL(_payText.rectTransform, 18, 128, 380, 30);
SquireAscentGame.Layout.cs:320 / void BuildStatus(RectTransform col)	_comfortText = T(k, "", 22, InkSoft); TL(_comfortText.rectTransform, 18, 156, 190, 30);
SquireAscentGame.Layout.cs:321 / void BuildStatus(RectTransform col)	_laurelText = T(k, "", 22, InkSoft, TextAnchor.MiddleRight); TL(_laurelText.rectTransform, 200, 156, 196, 30);
SquireAscentGame.Layout.cs:339 / void BuildChapterCard(RectTransform area)	_chNum = T(ribbon.transform, "Chapter 1", P ? 32 : 26, Ink, TextAnchor.MiddleCenter, true); _chNum.rectTransform.Stretch();
SquireAscentGame.Layout.cs:436 / void BuildLists()	label = T(bg.transform, text, P ? 32 : 25, InkSoft, TextAnchor.MiddleCenter, true);
SquireAscentGame.Layout.cs:459 / void BuildLists()	r.name = T(bg.transform, task.name, FName, Ink, TextAnchor.MiddleLeft, true);
SquireAscentGame.Layout.cs:462 / void BuildLists()	r.val = T(bg.transform, "", FSmall, InkSoft, TextAnchor.MiddleRight, true);
SquireAscentGame.Layout.cs:495 / void BuildLists()	r.name = T(bg.transform, it.name, FName, Ink, TextAnchor.MiddleLeft, true);
SquireAscentGame.Layout.cs:499 / void BuildLists()	r.lvl = T(bg.transform, "", P ? 28 : 20, InkSoft, TextAnchor.MiddleLeft);

`SquireAscentGame.Layout.cs:541 / void BuildLists()`	`var t = T(bg.transform, "", P ? 28 : 21, InkSoft);`
`SquireAscentGame.Layout.cs:561 / void BuildAgesPanel(RectTransform content)`	`_agesStory = T(c, "", P ? 30 : 23, Ink, TextAnchor.UpperCenter);`
`SquireAscentGame.Layout.cs:563 / void BuildAgesPanel(RectTransform content)`	`_rebirthInfo = T(c, "", P ? 28 : 21, InkSoft, TextAnchor.UpperCenter);`
`SquireAscentGame.Layout.cs:568 / void BuildAgesPanel(RectTransform content)`	`_agesStats = T(c, "", P ? 28 : 21, InkSoft, TextAnchor.UpperLeft);`
`SquireAscentGame.Layout.cs:574 / void BuildAgesPanel(RectTransform content)`	`_ascendInfo = T(grp, "", P ? 28 : 21, InkSoft, TextAnchor.UpperCenter);`
`SquireAscentGame.Layout.cs:620 / void RefreshTabs()`	`label.color = sel ? Ink : (locked ? new Color(0.8f, 0.8f, 0.8f, 0.6f) : Cream);`
`SquireAscentGame.Popups.cs:41 / void Enqueue(Action show)`	`msg = HzUI.Label("Msg", card.transform, text, textSize, Ink, TextAnchor.UpperCenter, false);`
`SquireAscentGame.Tutorial.cs:620 / void BuildCoach(CoachStep s)`	`var msg = HzUI.Label("Msg", bub.transform, s.text, fs, Ink, TextAnchor.UpperLeft, false);`
`SquireAscentGame.Tutorial.cs:644 / void BuildCoach(CoachStep s)`	`var b = HzUI.Button("Skip", bub.transform, "Skip tour", X("#B9A88C"), Ink, EndChain, P ? 34 : 26, 22);`

Source	Exact current code
`SqData.cs:50 / class/static declarations`	`public string color = "#888888";`
`SqData.cs:84 / class/static declarations`	`public string name = "", stat = "xp", target = "all", color = "#88CC88";`
`SquireAscentGame.cs:24 / static Color X(string h) => HzUI.Hex(h);`	`static readonly Color Parchment = X("#F3E3C3"), Parch2 = X("#E9D3A8"), Ink = X("#3B2A1E"), InkSoft = X("#6B5440"),`
`SquireAscentGame.cs:25 / static Color X(string h) => HzUI.Hex(h);`	`Teal = X("#1F4E5F"), TealDeep = X("#13303B"), Gold = X("#E2B33C"), Crimson = X("#C2453A"), Moss = X("#5E9B4A"),`
`SquireAscentGame.cs:26 / static Color X(string h) => HzUI.Hex(h);`	`Violet = X("#7E5BA6"), Sky = X("#3BA7C9"), Good = X("#3E9B4F"), Bad = X("#C2453A"), Locked = X("#9C8A72"), Cream = X("#FFF8EA");`

`SquireAscentGame.Layout.cs:79 / void BuildBackground()`	`var g = HzUI.Img("Bg", Root, HzUI.Gradient(X("#2B5B6B"), X("#0F2730")), Color.white);`
`SquireAscentGame.Layout.cs:273 / void BuildStatusPortrait(RectTransform hud)`	`var bar = Bar(c, X("#CDB68A"), Crimson, out _lifeFill, 14); TL(bar.rectTransform, 256, 176, 560, 28);`
`SquireAscentGame.Layout.cs:308 / void BuildStatus(RectTransform col)`	`var bar = Bar(c, X("#CDB68A"), Crimson, out _lifeFill, 12); HBand(bar.rectTransform, 184, 28, 16, 16);`
`SquireAscentGame.Layout.cs:342 / void BuildChapterCard(RectTransform area)`	`_chGoal = T(c, "", P ? 34 : 26, X("#F7E7C2"));`
`SquireAscentGame.Layout.cs:344 / void BuildChapterCard(RectTransform area)`	`var bar = Bar(c, X("#0C2027"), Gold, out _chFill, 12);`
`SquireAscentGame.Layout.cs:348 / void BuildChapterCard(RectTransform area)`	`_chHint = T(c, "", P ? 28 : 19, X("#D7E8EB"), TextAnchor.UpperLeft);`
`SquireAscentGame.Layout.cs:384 / void BuildBoosterBar(RectTransform bar, float size)`	`v.name = T(bar, b.name, P ? 22 : 15, X("#CFE3E6"), TextAnchor.MiddleCenter, true);`
`SquireAscentGame.Layout.cs:465 / void BuildLists()`	`var bar = Bar(bg.transform, X("#D9C59C"), catColor, out r.fillRt, P ? 14 : 11);`
`SquireAscentGame.Layout.cs:736 / void RefreshSelection()`	`r.bg.color = act ? X("#FFF4D6") : Cream;`
`SquireAscentGame.Layout.cs:893 / void RefreshItems()`	`r.bg.color = active ? X("#FFF4D6") : Cream;`
`SquireAscentGame.Layout.cs:971 / void RefreshAges()`	`_rebirthBtn.GetComponent<Image>().color = _sim.CanRebirth ? Teal : X("#8C9AA0");`
`SquireAscentGame.Layout.cs:982 / void RefreshAges()`	`_ascendBtn.GetComponent<Image>().color = _sim.CanAscend ? Sky : X("#8C9AA0");`
`SquireAscentGame.Popups.cs:206 / void ShowTwilight()`	`Enqueue(() => OpenCard("twilight", "Twilight", $"At {_sim.Age} your years have run out and time stands still.\nTurn the Hourglass of Ages to begin again, stronger.", X("#5B4A7A"), "pt_elder",`
`SquireAscentGame.Popups.cs:208 / void ShowTwilight()`	`(_sim.CanAscend ? "Ascend" : "Wait", _sim.CanAscend ? Sky : X("#8C9AA0"), _sim.CanAscend ? (Action)DoAscend : () => { Hint("twilight"); }));`

`SquireAscentGame.Popups.cs:215 / void ConfirmRebirth()`	`("Turn the hourglass", Teal, DoRebirth), ("Not yet", X("#8C9AA0"), null));`
`SquireAscentGame.Popups.cs:222 / void ConfirmAscend()`	`("Ascend", Sky, DoAscend), ("Not yet", X("#8C9AA0"), null));`
`SquireAscentGame.Tutorial.cs:73 / void ShowIntro(int page)`	`Teal, "pt_youth", ("Next", Moss, () => ShowIntro(1)), ("Skip the tour", X("#8C9AA0"), () => { MarkTut("intro"); MarkTut("tour"); Toast("Tap ? any time for How to play", Cream, 2f); }));`
`SquireAscentGame.Tutorial.cs:83 / void ShowIntro(int page)`	`X("#5B4A7A"), "ui_hourglass", ("Show me around", Moss, () => { MarkTut("intro"); QueueTour(); }));`
`SquireAscentGame.Tutorial.cs:109 / void ShowHowToPlay(int page)`	`title = "Skills"; art = "sk_focus"; accent = X("#4C5E9B");`
`SquireAscentGame.Tutorial.cs:117 / void ShowHowToPlay(int page)`	`title = "Ages"; art = "ui_hourglass"; accent = X("#5B4A7A");`
`SquireAscentGame.Tutorial.cs:129 / void ShowHowToPlay(int page)`	`OpenCard("howto", $"{title} 1/{HowPages}", text, accent, art, ("Next", Moss, () => ShowHowToPlay(1)), ("Close", X("#8C9AA0"), null));`
`SquireAscentGame.Tutorial.cs:290 / void UpdateAdvice(bool force = false)`	`_chHint.text = "<color=#E2B33C><b>Next step: </b></color> " + text;`
`SquireAscentGame.Tutorial.cs:644 / void BuildCoach(CoachStep s)`	`var b = HzUI.Button("Skip", bub.transform, "Skip tour", X("#B9A88C"), Ink, EndChain, P ? 34 : 26, 22);`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/SquireAscentGame.cs`, find the displayed literal `"Squire's Ascent"`, and replace its displayed value with `"Squire's Ascent Edition"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact helper names, unlocks, charges and effects: Resources/squireascent/Levels/levels.json / boosters. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`SqAssets.cs:63 / public static string ItemIcon(string id) => "it_" + id;`	`var l = HzUI.Label("L", disc.transform, string.IsNullOrEmpty(letter) ? "?" : letter.Substring(0, 1), (int)(size * 0.55f), Color.white);`
`SquireAscentGame.cs:121 / void Consume()`	`if (s.EvBankrupt) { SqSfx.Play(SqSfx.Bankrupt, 0.9f); Toast("Bankrupt! You lost your home and goods.", Bad, 2.2f); Hint("bankrupt"); }`
`SquireAscentGame.cs:129 / void Consume()`	`if (neg && !_warnedNegative && S.coins > 0) { _warnedNegative = true; SqSfx.Play(SqSfx.Warning, 0.7f); Toast("Careful: your costs are higher than your pay!", Bad, 2f); Hint("expenses"); }`
`SquireAscentGame.cs:193 / void OnBooster(string id)`	`if (!Unlocked(b) || !Granted(b)) { SqSfx.Play(SqSfx.Locked, 0.7f); Toast($"{b.name} unlocks at chapter {b.unlock}", Locked); ShakeBooster(id); return; }`
`SquireAscentGame.cs:195 / void OnBooster(string id)`	`if (_sim.Twilight) { SqSfx.Play(SqSfx.Locked, 0.6f); Toast("Time has stopped. Begin a New Age first.", Locked); return; }`
`SquireAscentGame.cs:196 / void OnBooster(string id)`	`if (b.duration > 0 && BoosterTime(id) > 0) { Toast($"{b.name} is already active", Cream); return; }`

`SquireAscentGame.cs:211 / void UseBooster(SqBooster b)`	`Toast("+" + SqFmt.Num(amt) + " crowns", Gold);`
`SquireAscentGame.cs:215 / void UseBooster(SqBooster b)`	`case "warp": S.warp = b.duration; SqSfx.Play(SqSfx.BoosterWarp, 0.9f); Toast("Time Warp! Days fly by twice as fast.", Sky); break;`
`SquireAscentGame.cs:216 / void UseBooster(SqBooster b)`	`case "focus": S.focus = b.duration; SqSfx.Play(SqSfx.BoosterFocus, 0.9f); Toast("Focus Candle! Skills train 3x faster.", Violet); break;`
`SquireAscentGame.cs:217 / void UseBooster(SqBooster b)`	`case "herald": S.herald = b.duration; _sim.HeraldStep(); SqSfx.Play(SqSfx.BoosterHerald, 0.9f); Toast("The Herald manages your career for 3 minutes.", Gold); break;`
`SquireAscentGame.cs:249 / void OnItem(int i)`	`SqSfx.Play(SqSfx.Locked, 0.6f); Toast("You can't afford to live there yet.", Bad); return;`
`SquireAscentGame.cs:261 / void OnItem(int i)`	`SqSfx.Play(SqSfx.Locked, 0.6f); Toast("That would cost more than you earn.", Bad); return;`
`SquireAscentGame.cs:278 / void DoRebirth()`	`Toast($"A New Age begins. Legacy bonuses grow!", Gold, 2f);`
`SquireAscentGame.cs:288 / void DoAscend()`	`Toast($"Ascended! +{SqFmt.Num(gain)} starlight", Sky, 2.2f);`
`SquireAscentGame.cs:387 / public override string DebugCommand(string cmd)`	`case "advice": return _chHint ? _chHint.text : "none";`
`SquireAscentGame.Layout.cs:98 / void BuildBackground()`	`var t = HzUI.Label("T", parent, text, size, color, align, display);`
`SquireAscentGame.Layout.cs:172 / void BuildLandscape()`	`MakeTabButton(tabs, 2, "Shop", "", 0, 0.5f, 76);`
`SquireAscentGame.Layout.cs:173 / void BuildLandscape()`	`MakeTabButton(tabs, 3, "Ages", "", 0.5f, 1, 76);`
`SquireAscentGame.Layout.cs:278 / void BuildStatusPortrait(RectTransform hud)`	`IconBtn(c, "help", HzIcons.Help, Teal, ShowHelp, 84).GetComponent<RectTransform>().Place(new Vector2(1, 1), new Vector2(84, 84), new Vector2(-58, -154));`

SquireAscentGame.Layout.cs:296 / void BuildStatus(RectTransform col)	IconBtn(head.transform, "help", HzIcons.Help, Teal, ShowHelp, 60).GetComponent<RectTransform>().Place(new Vector2(1, 0.5f), new Vector2(60, 60), new Vector2(-104, 0));
SquireAscentGame.Layout.cs:364 / void BuildBoosterBar(RectTransform bar, float size)	var btn = HzUI.Button("Booster_" + b.id, bar, null, Teal, Color.white, () => OnBooster(id), 40, (int)(size * 0.3f));
SquireAscentGame.Layout.cs:553 / void BuildAgesPanel(RectTransform content)	tl.text = $"The Hourglass of Ages appears at age {_file.config.agesTabAge:0}.";
SquireAscentGame.Layout.cs:565 / void BuildAgesPanel(RectTransform content)	_rebirthBtn = HzUI.Button("Rebirth", c, "Begin a New Age", Teal, Color.white, ConfirmRebirth, P ? 40 : 32, 26);
SquireAscentGame.Layout.cs:576 / void BuildAgesPanel(RectTransform content)	_ascendBtn = HzUI.Button("Ascend", grp, "Ascend", Sky, Color.white, ConfirmAscend, P ? 40 : 32, 26);
SquireAscentGame.Layout.cs:589 / void OnTabClick(int t)	if (t == 2 && !S.shopOpen) { SqSfx.Play(SqSfx.Locked, 0.6f); Toast($"The market opens once you hold {SqFmt.Num(_file.config.shopCoins)} crowns.", Locked); return; }
SquireAscentGame.Layout.cs:590 / void OnTabClick(int t)	if (t == 3 && !S.agesOpen) { SqSfx.Play(SqSfx.Locked, 0.6f); Toast($"Something stirs at age {_file.config.agesTabAge:0}...", Locked); return; }
SquireAscentGame.Layout.cs:650 / void RefreshStructure()	_shopLockedText.text = $"The market opens once you hold {SqFmt.Num(_file.config.shopCoins)} crowns.";
SquireAscentGame.Layout.cs:753 / void RefreshHud()	if (k != _hudKeyNet) { _hudKeyNet = k; _netText.text = (net >= 0 ? "+" : "") + SqFmt.Num(net) + " / day"; _netText.color = net >= 0 ? Good : Bad; }
SquireAscentGame.Layout.cs:755 / void RefreshHud()	if (k != _hudKeyPay) { _hudKeyPay = k; _payText.text = "Pay " + SqFmt.Num(_sim.IncomeNow) + " Costs " + SqFmt.Num(_sim.Expense); }
SquireAscentGame.Layout.cs:757 / void RefreshHud()	if (k != _hudComfort) { _hudComfort = k; _comfortText.text = "Comfort " + SqFmt.Mult(_sim.Comfort); }

SquireAscentGame.Layout.cs:759 / void RefreshHud()	if (k != _hudLaurel) { _hudLaurel = k; _laurelText.text = "Laurels " + SqFmt.Mult(_sim.LaurelMult); }
SquireAscentGame.Layout.cs:764 / void RefreshHud()	_ageText.text = "Age " + age + " Day " + day;
SquireAscentGame.Layout.cs:771 / void RefreshHud()	if (_lifeText.text.Length == 0 || age != _lastLifeAge || life != _lastLife) { _lastLifeAge = age; _lastLife = life; _lifeText.text = $"{age} / {life} years"; }
SquireAscentGame.Layout.cs:782 / void RefreshHud()	_activeLine.text = "Training " + _sim.Task(S.skill).name;
SquireAscentGame.Layout.cs:834 / void RefreshRows(bool force = false)	r.lvl.text = S.maxLvl[t] > 0 ? $"Lv {lv} best {S.maxLvl[t]}" : "Lv " + lv;
SquireAscentGame.Layout.cs:842 / void RefreshRows(bool force = false)	if (_sim.IsJob(t)) r.val.text = "+" + SqFmt.Num(v);
SquireAscentGame.Layout.cs:843 / void RefreshRows(bool force = false)	else r.val.text = SqFmt.Mult(v) + " " + StatName(_sim.Task(t));
SquireAscentGame.Layout.cs:889 / void RefreshItems()	r.lvl.text = eff + " " + (it.expense > 0 ? "-" + SqFmt.Num(_sim.ItemCost(i)) + "/d" : "free");
SquireAscentGame.Layout.cs:890 / void RefreshItems()	r.state.text = home ? (active ? "Home" : "Move in") : (active ? "Owned" : "Buy");
SquireAscentGame.Layout.cs:908 / void UpdateChapterProgress()	if (Math.Abs(SqFmt.Key(shown) - _chLast) > 0.5) { _chLast = SqFmt.Key(shown); _chProg.text = c.kind == "home" ? (cur >= 1 ? "Done" : "Not yet") : SqFmt.Num(shown) + " / " + SqFmt.Num(target); }
SquireAscentGame.Layout.cs:915 / void RefreshChapter()	_chNum.text = "Chapter " + (S.chapter + 1);
SquireAscentGame.Layout.cs:916 / void RefreshChapter()	_chTitle.text = (c.breather ? "Breather: " : "") + c.title;
SquireAscentGame.Layout.cs:917 / void RefreshChapter()	_chGoal.text = "Goal: " + SqSim.GoalText(c, _sim);
SquireAscentGame.Layout.cs:935 / void RefreshBoosters()	v.badge.text = ch > 0 ? ch.ToString() : "+";

`SquireAscentGame.Layout.cs:949 / void RefreshBoosterTimers()`	`if (v.lastTime > 0 && sec == 0) { SqSfx.Play(SqSfx.BoosterEnd, 0.6f); Toast(b.name + " has ended", Cream, 1.2f); }`
`SquireAscentGame.Layout.cs:952 / void RefreshBoosterTimers()`	`if (sec > 0) v.timer.text = sec >= 60 ? (sec / 60) + ":" + (sec % 60).ToString("00") : sec + "s";`
`SquireAscentGame.Layout.cs:984 / void RefreshAges()`	`_agesStats.text = $"New Ages begun: {S.rebirths}\nAscensions: {S.ascensions} Starlight: {SqFmt.Num(S.starlight)}\nLaurels: {S.chapter} ({SqFmt.Mult(_sim.LaurelMult)} all XP)\nTotal level this life: {_sim.TotalLevels}\nTime played: {SqFmt.Time(S.playSeconds)}";`
`SquireAscentGame.Popups.cs:41 / void Enqueue(Action show)`	`msg = HzUI.Label("Msg", card.transform, text, textSize, Ink, TextAnchor.UpperCenter, false);`
`SquireAscentGame.Popups.cs:53 / void Enqueue(Action show)`	`var t = HzUI.Label("Title", ribbon.transform, title, P ? 60 : 54, Color.white); t.rectTransform.Stretch(8);`
`SquireAscentGame.Popups.cs:136 / void Hint(string key)`	`OpenCard("hint_" + key, h.title, h.text, Teal, null, ("Got it", Moss, null));`
`SquireAscentGame.Popups.cs:188 / void ShowOutOfCharges(SqBooster b)`	`Toast("+1 " + b.name, Gold);`
`SquireAscentGame.Tutorial.cs:73 / void ShowIntro(int page)`	`Teal, "pt_youth", ("Next", Moss, () => ShowIntro(1)), ("Skip the tour", X("#8C9AA0"), () => { MarkTut("intro"); MarkTut("tour"); Toast("Tap ? any time for How to play", Cream, 2f); }));`
`SquireAscentGame.Tutorial.cs:290 / void UpdateAdvice(bool force = false)`	`_chHint.text = "<color=#E2B33C><b>Next step: </b></color> " + text;`
`SquireAscentGame.Tutorial.cs:333 / void OnSettings()`	`p.Buttons[2].SetText("How to play");`
`SquireAscentGame.Tutorial.cs:334 / void OnSettings()`	`p.Buttons[3].SetText("Replay tutorial");`
`SquireAscentGame.Tutorial.cs:425 / void QueueTour()`	`onEnd = () => { if (P) SetTab(0); SqSfx.Play(SqSfx.Unlock, 0.6f); Toast("Good luck, " + _sim.Task(S.job).name + "!", Gold, 1.8f); },`

`SquireAscentGame.Tutorial.cs:620 / void BuildCoach(CoachStep s)`	`var msg = HzUI.Label("Msg", bub.transform, s.text, fs, Ink, TextAnchor.UpperLeft, false);`
`SquireAscentGame.Tutorial.cs:637 / void BuildCoach(CoachStep s)`	`var b = HzUI.Button("Next", bub.transform, label, Moss, Color.white, () => { if (s.next != null) AdvanceCoach(); else EndChain(); }, P ? 40 : 30, 22);`
`SquireAscentGame.Tutorial.cs:644 / void BuildCoach(CoachStep s)`	`var b = HzUI.Button("Skip", bub.transform, "Skip tour", X("#B9A88C"), Ink, EndChain, P ? 34 : 26, 22);`
`SquireAscentGame.Tutorial.cs:748 / string CoachDebug()`	`return $"{_coach.key} wait={_coach.waitEvent} next={_coach.next} target={(t ? ScreenXY(t) : "none")} q={_coachQ.Count} text={_coach.text}";`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/squireascent through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/squireascent/Sfx. Preserve each actual load key and importer settings when replacing sounds. This game has no supplied music loop under its Resources Music folder; do not assume that a Music toggle is proof of a bundled track. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.squireascent`, version, scene orientation and signing. The build command writes `Builds/Android/squireascent.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/squireascent/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.